

the gurky doc

Method	unsorted LinkedList	sorted ExtensibleList	Heap
PQ.insert()	$\Theta(1)$	$\mathcal{O}(n)$	$\mathcal{O}(\log(n))$ (amo.)
PQ.removeMin()	$\mathcal{O}(n)$	$\Theta(1)$	$\mathcal{O}(\log(n))$
PQ.min()	$\mathcal{O}(n)$	$\Theta(1)$	$\Theta(1)$
APQ.remove()	$\Theta(1)$	$\Theta(1)$	$\mathcal{O}(\log(n))$
APQ.replaceKey()	$\Theta(1)$	$\mathcal{O}(n)$	$\mathcal{O}(\log(n))$
APQ.replaceValue()	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$

*APQ<Key, Value>

Method	ELGraph (DS 2.5.24)	ALGraph (DS 2.5.25)	AMGraph (DS 2.5.26)
Memory usage (all)	$V + E$	$V + E$	V^2
outgoingEdges($v \in \text{Vertex}$),	E	$\deg(v)$	V
incomingEdges($v \in \text{Vertex}$)	E	$\deg(v)$	V
getEdge($u, v \in \text{Vertex}$)	E	$\min\{\deg(u), \deg(v)\}$	1
insertVertex(x)	1	1	V^2
insertEdge(u, v, x)	1	1	1
removeVertex(v)	E	$\deg(v)$	V^2
removeEdge(e)	1	1	1

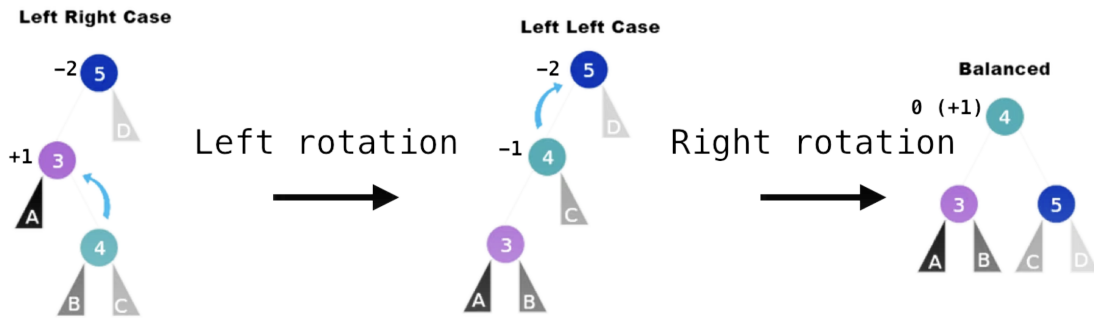
	In-place	stable	Worst	Avg	Best
Selection	yep	nope	n^2	n^2	n^2
Insertion	yep	yep	n^2	n^2	n^2
Merge (parallelisable)	nope	yep	$n \log(n)$	$n \log(n)$	$n \log(n)$
Quick	maybe	nope	n^2	$n \log(n)$	$n \log(n)$
Bubble	yep	yep	n^2	n^2	n
Heap	maybe	nope	$n \log(n)$	$n \log(n)$	$n \log(n)$
Bucket	nope	yep	$n + N$	$n + N$	$n + N$
Lexicographic	nope	yep	$d \cdot T(n)$	$d \cdot T(n)$	$d \cdot T(n)$
Radix	nope	yep	$d \cdot (n + N)$	$d \cdot (n + N)$	$d \cdot (n + N)$

$d := \text{number of components in each tuple}$

Definition (complete tree): $T \in \text{BinTree}$ is *complete* iff every level except the bottom level is full, and all leaves are as leftmost as possible.

Definition (proper tree, full tree): $T \in \text{BinTree}$ is *proper* (aka *full*) iff every non-leaf node of T has exactly two children.

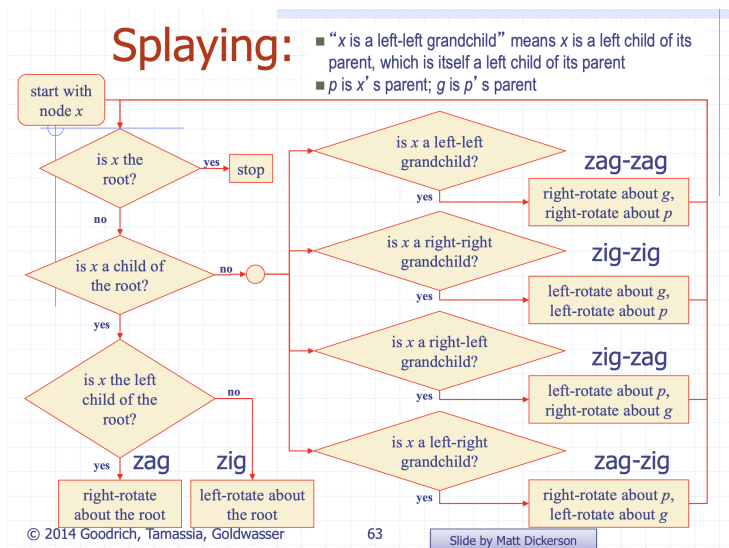
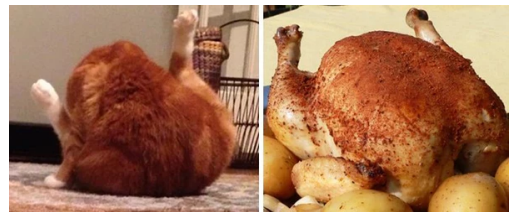
AVL Balance — Height = $O(\log N)$



Dijkstra's	$O(V + E)\log(V)$
Primmy	$O(V + E)\log(V)$
Kruskal's	$O(E)\log(V)$

	AVL	Red Black
Balance	stricter	not
search	faster	slower
inseart	slower	faster
remove	slower	faster

B-Tree	get, put, remove
shallow Tree	$O(h)$
short searches, good for DB	replaced by in-order traversal child



Goal for collision handling: still store the key, but do it in a time- and space-efficient way. Some strategies are, where k is the key stored:

Separate chaining: Each entry in the table stores a *list* of `Entry<Key, Value>` items.

- Lookups are now far more expensive as more hash collisions occur.
- The list may be sorted, which makes insertion slower but lookups faster.

Probing: Iterate for $i = 0, 1, \dots, N-1$, and “probe” the indices $f(k, i)$ (f will be defined below).

Store into the first empty entry.

- Lookups are now more expensive.
- So is insertion.

Linear: $f(k, i) = (h(k) + i) \bmod N$

Quadratic: $f(k, i) = (h(k) + i^2) \bmod N$

Double hashing: $f(k, i) = (h(k) + id(k)) \bmod N$, where d is another hash function

Hashing performance: for any of `get()`, `put()`, `remove()`:

- *Expected* runtime: $\Theta(1)$
- *Worst* runtime: $\mathcal{O}(n)$ (if all keys collide to the same value)

The **load factor** is $\alpha = \frac{n}{N} = \frac{\text{size}}{\text{capacity}}$. The expected number of probes we need will be $\frac{1}{1-\alpha}$.

Acceptable load factors:

- Separate chaining: $\alpha \in [0.8, 1.0]$
- Probing: $\alpha \in [0, 2/3)$

Algorithm 29: Boyer-Moore (problem 3.2.2 solver)

```

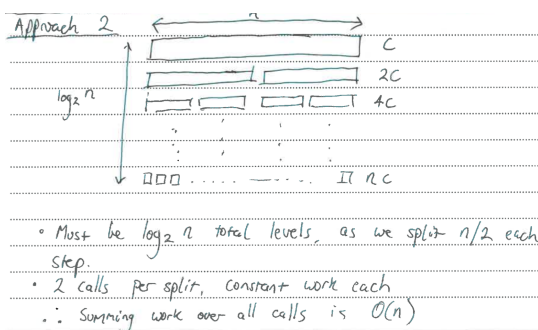
1 method boyerMoore( $T \in \text{String}\langle \mathcal{A} \rangle, P \in \text{String}\langle \mathcal{A} \rangle, \mathcal{A} \in \text{Alphabet}$ )
   $\rightarrow \mathbb{Z}_{\geq 0} \cup \{\text{NO\_MATCH}\}$ 
  Input : A pattern  $P$  to find in a text  $T$ , both of which are strings over an alphabet  $\mathcal{A}$ 
  Returns : The smallest  $i \in \mathbb{Z}_{\geq 0}$  such that  $P = T[i, \dots, i + \text{len}(P) - 1]$ , or NO_MATCH if there is no such  $i$ 
  Runtime:  $\mathcal{O}(\text{len}(P) \cdot \text{len}(T) + |\mathcal{A}|)$  worst
2   $L \leftarrow \text{lastOccuranceFunction}(P, \mathcal{A});$ 
3   $t \leftarrow \text{len}(P) - 1;$ 
4   $p \leftarrow \text{len}(P) - 1;$ 
5  do
6    if  $T[t] = P[p]$  then // this character matches
7      if  $p = 0$  then // whole pattern matches
8        return  $i;$  // match at index  $i$ 
9      else
10        $t \leftarrow t - 1;$ 
11        $p \leftarrow p - 1;$ 
12    else
13      // Character jump
14       $t \leftarrow t + \text{len}(P) - \min\{p, 1 + L.\text{get}(T[t])\};$ 
15       $p \leftarrow \text{len}(P) - 1;$ 
16  while  $t < \text{len}(T);$ 
17  return NO_MATCH;
```

Algorithm 30: Knuth-Mogus-Pratt (problem 3.2.2 solver)

```
1 method knuthMorrisPratt( $T \in \text{String}\langle\mathcal{A}\rangle, P \in \text{String}\langle\mathcal{A}\rangle$ )  $\rightarrow \mathbb{Z}_{\geq 0} \cup \{\text{NO\_MATCH}\}$ 
   Input   : A pattern  $P$  to find in a text  $T$ , both of which are strings over an alphabet  $\mathcal{A}$ 
   Returns : The smallest  $i \in \mathbb{Z}_{\geq 0}$  such that  $P = T[i, \dots, i + \text{len}(P) - 1]$ , or NO MATCH if there is no such  $i$ 
   Runtime:  $\Theta(\text{len}(T) + \text{len}(P))$  worst

2  $F \leftarrow \text{failureFunction}(P)$ ;
3  $t \leftarrow 0$ ;
4  $p \leftarrow 0$ ;
5 while  $t < \text{len}(T)$  do
6   if  $T[t] = P[p]$  then // this character matches
7     if  $p = \text{len}(P) - 1$  then // matched whole string
8       return  $t - p$ ;
9     else
10      // Keep looking...
11       $t \leftarrow t + 1$ ;
12       $p \leftarrow p + 1$ ;
13   else
14     // Mismatch...
15     if  $p > 0$  then
16       // Shift over pattern as much as possible
17        $p \leftarrow F[p - 1]$ ;
18     else
19       // Try next character of target
20        $t \leftarrow t + 1$ ;
21 return NO MATCH
```

$$\begin{aligned} T(n) &= O(1) : n = 1 \\ &= 2T(n/2) + O(1) : n > 1 \end{aligned}$$



$$\begin{aligned} &2T(n/2) + C \\ &= 2[2T(n/4) + C] + C \\ &= 4T(n/4) + 3C \\ &= 2[2[2T(n/8) + C] + C] + C \\ &= 8T(n/8) + 7C \\ &\dots = 2^k T(n/2^k) + (2^k - 1)C \\ &\text{Want } T(n/2^k) = T(1) \Rightarrow k = \log_2 n \\ &\Rightarrow 2^{\log_2 n} T(n/2^{\log_2 n}) + (2^{\log_2 n} - 1)C \\ &= nT(n/n) + (n - 1)C \\ &= nC + (n - 1)C = O(n) \end{aligned}$$